

Philipps-Universität Marburg
Fachbereich Mathematik und Informatik

Bachelor's thesis

User Stories Quality Analyse Using Secure Generative AI

Dependency and Independency between User Stories

Thi Minh Hanh Nguyen
3393390

July 4, 2025

Examiner: Gabi
Advisor: Lukas

Contents

1 Motivation **3**
- 1.1 Agile Development and User Stories . . . 3
- 1.2 Quality of User Stories . . . 3
- 1.3 Importance of User Story's Dependency . . . 4
- 1.4 Related work . . . 4

2 Fundamentals **5**
- 2.1 User Story and its conceptual model . . . 5
  - 2.1.1 User Story . . . 5
  - 2.1.2 Conceptual Model . . . 5
  - 2.1.3 Inherited notation and quality definitions of user stories . . . 6
- 2.2 Definition of Dependency and Independency . . . 6
  - 2.2.1 Independency . . . 6
    - Requirement independency . . . 6
    - User Story independency . . . 6
  - 2.2.2 Dependencies . . . 7
    - Requirement dependency types . . . 7
    - User Stories dependency types . . . 8

3 Methodology **14**
- 3.1 Dataset . . . 14
- 3.2 PoC . . . 14
- 3.3 Evaluation . . . 14

Bibliography **15**

1 Motivation

1.1 Agile Development and User Stories

How important is agile development and so that the importance of user stories? implies the quality of user stories can affect the quality of the software quality.

In requirement engineering, agile development is a widely adopted method to enhance the waterfall method of software development. Agile development enables rapid development: It allows for iterative, small-scale, and rapid development and functionality delivery, leading to a rapid response to user requirements [5]. In the artificial intelligence era, a software prototype can be deployed using agile development for requirement analysis in no time.

Agile Manifesto:The Agile Manifesto values (Individuals and interactions, Working software, Customer collaboration, Responding to change) are presented as better ways of developing software compared to the values of traditional approaches (processes and tools, comprehensive documentation, contract negotiation, following a plan).[5]

Scrum is the widely adopted agile framework[14]. Scrum contains roles, artifacts, and processes[5]. There are many layers of artifacts in SCRUM such as Epic, Feature, User Stories, and Product Backlog. These are textual high-level user requirements in SCRUM.

User stories, which are fundamental for the successful software development, are the starting base of agile frameworks[5]. Shifting-left of the quality check is needed to control software quality in early phases and to increase customer satisfaction.

1.2 Quality of User Stories

US needs to capture all essential elements of a user requirement. In order to evaluate the quality of USs, many researchers have already defined different quality frameworks to check the quality not only user requirements but also of a US and also a set of USs.

Leffingwell et al. defined a set of criteria called INVEST: Independent, Negotiable, Valuable to users or customers, Estimable, Small, and Testable.[7]

Lai defined three factors for evaluating US quality: Basic Quality (Discussable and Estimable), Management Quality (Controllable and Manageable), and Acceptance Quality (Confirmable). Each factor consists of three metrics, with different weights, and the final quality metric is the weighted sum of all three factors. They also suggested rules for correcting low-quality USs.[6]

Lucassen et al. presented the Quality User Story (QUS) framework, which is a collection of 14 rules that determine the quality of not only a US but also a set of USs.

Using this framework, they developed a tool called AQUSA, which uses NLP techniques to verify some of the QUS criteria automatically [11]. Each quality rule is classified according to three concepts from linguistics: syntactic, semantic, and pragmatic.The QUS framework and the AQUSA tool have been combined into a package called the Grimm Method [9].

Femmer et al. defined low-quality criteria of requirements as "requirement smells" [2], a concept inspired by Code Smells in software engineering. A prototype tool called *Smella* was developed to detect these quality issues automatically.

Xu et al. [15] introduced a user requirement quality model, which consists of 6 quality criteria for individual user requirements (Complete, Consistent, Simple, Accurate) and overall user requirements (Robust, Unique, Harmonious). User requirements are represented by USs.

1.3 Importance of User Story's Dependency

Why dependencies between user stories is important to tracing and marked?

1.4 Related work

1. A Systematic and Lightweight Method to Identify Dependencies between USs
2. User Story Quality Assessment Based on Multi-dimensional Perspective: A Preliminary Framework
3.

2 Fundamentals

2.1 User Story and its conceptual model

2.1.1 User Story

Leffinwell et.al defined in "A User Story Primer" a User Story as "a brief statement of intent that describes something the system needs to do for the user."[7]

Definition 2.1 — User story (US) is a user-centric natural language software requirement to describe a software feature. US is semi-structuredly notated when it follows a prescribed format (template). This template helps user stories captures key aspects: *who* wants the functionality, *what* functionality the end-users expect, and optionally why the end-users need this functionality.

There is no universal template of user stories but the mostly used is the Connextra template [10]: *As a <role>, I want <goal>,[so that <benefit>]"*. Or the other terminology is This kind of template defines the high-level representation of US. For further automatically quality analysis of USs, a more granular blueprint of US's structure is needed.

2.1.2 Conceptual Model

Hoffman et.al defined conceptual model like structured representation of user stories (USs) [4].

This definition is inspired by the informal definitions of Hoffman et.al and Arulmohan et.al.

Definition 2.2 — A conceptual model of US captures US in conceptual schema that defines key components of a US and their relationships. The conceptual model can be illustrated by using diagrams such as UML diagrams (e.g. class diagram) in order to describe the relations between entities.

Since the researches of US started, there are some conceptual models derived from the Connextra template. Different level of abstraction can affected the conceptual model of a US.
Lucassen et al. had defined a conceptual models with 3 parts of a US, which are ROLE, MEANS, ENDS [12]. Each US has one ROLE, one MEANS, and zero to one ENDS. Each ROLE contains one indicator and one functional role. Each MEANS consists of one subject, one main object, one main verb, and zero to one free form. At last, the ENDS has the similar structure to the MEANS [12]. In other paper of the same authors,

they introduced that US contains 4 parts one role, one means, zero or more ends, and one format [11]. The second conceptual model is more detailed. This points out that conceptual model of USs can have different level of abstraction.
Arulmohan et al. also defined a metamodel for backlogs [1]. A *Backlog* contains one to N *US*, one to N *Persona*, and one to N *Entity*. A *US* consists of one *Persona*, one *Entity*, one *Action* , and zero to one *Benefit*. There are 2 one-side relations: a `trigger`-relation from *Persona* to *Action* and a `targets`-relation from *Action* to *Entity*.
Hoffmann et al. adopted introduced metamodel of Arulmohan et al. to define the conceptual model for anotated USs [4]. The conceptual model structures user stories using annotation graphs, defining elements like *Persona, GoalAction, GoalEntity, BenefitAction, and BenefitEntity*. These components are connected by relationships, helping identify strict and semantic redundancies in agile development.
What is the advantage of using this conceptual model? Does the conceptual model depend on the dataset?

2.1.3 Inherited notation and quality definitions of user stories

This thesis is an extension of the paper of Hofmann et al. [4]. This bachelor thesis will focus on detecting dependency and independency between USs. These are qualities of a set USs.

2.2 Definition of Dependency and Independency

2.2.1 Independency

Requirement independency

There are not many papers regarding the dependencies between USs, so it is rational to firstly take a look into the requirement dependencies. On the requirement level, there are some papers which already automated the relation extraction using NLP and Machine Learning. Guan et al. [3] proposed an automatic approach to extract requirement dependency based on integrated active learning strategies. They defined "basis dependencies" and "advanced dependencies." Firstly, a pair of requirements can be checked with "basis dependencies", this means that this pair is either dependent or independent (XOR-relation). Then, if this pair is dependent, specific dependency types can be extracted, which are *Notification, Arouse, Call, Conflict, Aggregation, and Similar.* So they defined *independency* between **two** requirements as follow: Two requirements (R_x, R_y) are independent if no recognized dependency type (e.g., notification, arouse, call, conflict, aggregation, similar) exists between them; otherwise, they are considered dependent. The definition of 6 types of dependency will be cited in the below paragraph.

User Story independency

Some researchers has specifically contributed to this topic but only applied on the USs. Leffingwell et al. [7] defined *Independency* in "A User Story Primer" that a US can be

developed, tested, and potentially delivered on its own, without being tied to other USs. This allows for more flexibility in prioritization and avoids dependencies that could slow down development. Ideally, each story should bring value on its own and be able to stand alone without requiring the completion of another story beforehand.
Lucassen et al. defined in their framework *Independency* based on the INVEST framework of Leffingwell et al. above. "User Stories should not overlap in concept and should be schedulable and implementable in any order".[11]
So based on the above researches, a definition of independency of a pair of USs is developed.

Definition 2.3 — A pair of USs (US_i, US_j) is called independent if there is no relation in the form of dependent semantic association between them, so that each of them can be implemented, tested for its fulfillment without affecting the other one.

For example, US_1 and US_2 are independent.
US_1: As a user, I want to change my profile picture.
US_2: As a user, I want to modify my address.

2.2.2 Dependencies

Requirement dependency types

Different papers with different terminology of dependency types in requirements engineering. For example:,

Guan et al. [3] developed a terminology that differentiates the six dependency types quote to quote as following: *Notification*, *Arouse*, *Call*, *Conflict*, *Aggregation*, *Similar*.

Motger et.al [13] introduced 6 relation types, which define how requirements depend on or affect each other. A *Requires* relation means one requirement must be fulfilled before another can proceed, while *Conflicts* shows that one requirement restricts another. If two requirements are mutually exclusive, they *Contradict* each other. *Is_a_variant* marks alternative ways to meet the same goal, and *Is_similar* highlights overlapping content or redundancy. Finally, *Details* means one requirement elaborates on another. These relations help structure requirements clearly, minimizing contradictions and overlaps to apply NLP methods.
Zang et.al [16] mentioned that various dependency terminologies has been proposed with different level of abstraction. They points out that there are some problems with existing dependency models that they have overlapping and vague types, some are hard to apply in practice and different users interpret them differently. These issues reduce clarity and usefulness, so the authors proposed a simpler, clearer model with better definitions. Dependencies are classified into 2 broad categories: intrinsic dependencies and additional dependencies. Intrinsic dependencies reflect the essential relationships that exist among requirements and directly impact various software engineering activities and most important they have clear affects at the implementation level, which are *Constrain:* This represents one requirement constraining another, often affecting implementation decisions or crosscutting concerns. *Precede:* This indicates a sequencing relationship

where one function must precede another, influencing process flows or control structures in the code. *Be_similar_to:* This highlights requirements that share similar data or behavior, often leading to reuse or shared components in implementation. *Refine:* This reflects hierarchical relationships where a higher-level requirement is broken down into more detailed sub-requirements, influencing module or class design. *Be_exception_of:* This identifies exception handling scenarios, which are critical for defining error-handling paths in the code. *Conflict:* This shows where the implementation of one requirement may negatively impact another, requiring careful architectural decisions to resolve. On the other hand, additional dependencies represent relationships that are introduced by specific software engineering tasks such as product release planning or cost-value trade-off analysis.
Li et.al[8] researched and consolidated 23 dependency types into a dependency model, which contains 3 main groups. `Document-related dependencies` arise from the relationships and structure within requirement documents. `Value-related dependencies` indicate how the fulfillment of one requirement influences the perceived value or cost of another, helping prioritize requirements in release planning. `Implementation-related dependencies` focus on how requirements are technically realized and how their implementations depend on each other, particularly relevant for change impact analysis during software design and development phases. The authors decided to narrow the scope to implementation-related dependencies because they wanted to focus on the most impactful and traceable relationships, which are either revealed by BPMN. Five implememtation-related dependency types are defined as following: "

- *Constraints:* A requirement can relate to another by being a constraint to the latter.
- *Refines:* One requirement can be refinement of another requirement, providing more detailed descriptions for the latter
- *Precondition:* Only after one function prescribed by a requirement is finished, or one condition described by the requirement is satisfied, that another function prescribed by a requirement can be performed. Usually precondition reflects the business rules or the sequence relationship between (sub-)processes.
- *Satisfies:* This type expresses that if one requirement is implemented in an application, the implementation will also satisfy another requirement.
- *Similar_to:* The prescription of one requirement is similar to or overlapping with one or more requirements.

"

What we learned to apply to better define user stories dependencies?

User Stories dependency types

Leffingwell et al. mentioned that many USs have some "natural, sequential dependencies as the product functionality builds, and yet each piece can deliver value independently." [7]
Lucassen et al.[11] emphasized that some dependencies between USs are "practically

impossible" to resolve because, analogous to the coupled system, some USs are linked to each other. They recommended establishing **explicit** dependencies, which can be solved by using issue tracker tools such as Jira and Azure DevOps. The dependencies of a US should be included as a link in the issue tracker tool. They also defined 2 types of US dependency relation between the two USs : *Causality* and *Superclasses*. A US_a is causally depended on US_b, if the US_a is "necessary" for starting development of the US_b. And a US_c is a *superclasses* of the US_d, when US_c is parent of US_d, which also means that US_c contains US_d.
For dependency management, some common agile project management tools e.g., Jira, Azure DevOps already integrated some default dependency types (linking). Such as in Jira, there are 4 default types *relates to, duplicates, blocks, clones* [1]. Azure DevOps also uses 5 common link types: *Affects, Duplicate, Parent/Child, Predecessor/Successor, Related*[2]
Only concentrate on functional User Stories, uses notion of Hoffmann to concrete define. Inspiration of xxx

Definition 2.4 (**Operation-based Dependency (OBD) - Pairwise**) — Let U be a set of user stories, and let $US_i, US_j \in U$.

Let $A(US_i)$ and $A(US_j)$ be the sets of activities associated with US_i and US_j, respectively, where each activity $a \in A(US_i)$ (and $b \in A(US_j)$) is defined as a pair:

$$a = (a.\text{Action.name},\ a.\text{Entity.name}) \qquad b = (b.\text{Action.name},\ b.\text{Entity.name})$$

Specifically, *Action* denotes a *GoalAction* and *Entity* denotes a *GoalEntity*.
Define the following:

- requires(a.Action.name, b.Action.name): One operation (e.g., delete, view, share) described in Action.name can only be implemented or executed if other operation (e.g., create, add) has been performed. This expresses a strict dependency where the prerequisite operation provides the necessary existence or state of the entity required for subsequent actions.

- a.Action.name $\approx$ b.Action.name: Lexical identity or semantic equivalence between `name`-attributes of Action of a and b.

An *Operation-based Dependency* exists between US_i and US_j **if and only if**

$$\exists\,(a,b) \in A(US_i) \times A(US_j) :\ a.\text{Entity.name} \approx b.\text{Entity.name} \\ \wedge\ \text{requires}\,\big(b.\text{Action.name},\ a.\text{Action.name}\big)$$

In words: An *operation-based dependency* between two user stories exists when an operation described in activity (primary action in Goal part only) in one story, such as a CRUD operation (Create, Read, Update, Delete) or another operation like share, view, or export, can only be carried out if the operation described in activity (primary action

[1] *Configuring issue linking*, 27.06.2025. Jira documentation
[2] *Reference guide for link types*, 27.06.2025. Azure DevOps Learn

in Goal part only) in the other story (e.g., create or add) has already been performed on the identical or a semantically equivalent entity. This means the one user story relies on the other one to ensure the necessary existence or state of the entity before its own action can proceed.

Definition 2.5 (**Sequential Dependency - Pairwise**) — Let U be a set of user stories, and let $US_i, US_j \in U$.

Let $A(US_i)$ and $A(US_j)$ denote the sets of activities associated with US_i and US_j, respectively. Let $R(US_i)$ and $R(US_j)$ denote the sets of reasons associated with US_i and US_j, respectively.

Each activity or reason $a \in A(US_i) \cup R(US_i)$ (and $b \in A(US_j) \cup R(US_j)$) is defined as a pair:

$$a = (a.\text{Action.name},\ a.\text{Entity.name}) \qquad b = (b.\text{Action.name},\ b.\text{Entity.name})$$

Define the set:

$$S := (A(US_i) \cup R(US_i)) \times (A(US_j) \cup R(US_j))$$

Define the following predicates:

- permits(a, b): One activity or reason establishes a necessary system service, condition, environment, configuration, or infrastructure such that other activity or reason can be meaningfully performed or deployed.
- OBD(a, b): There exists an Entity Enabling Constraint between a and b (as previously defined).
- $a \approx b$: a and b are lexically identical or semantically equivalent, i.e., both pairs have `name`-attributes of both Action and Entity that are lexically identical or semantically equivalent; that is,

$$a.\text{Action.name} \approx b.\text{Action.name} \ \wedge\ a.\text{Entity.name} \approx b.\text{Entity.name}$$

A *Sequential Dependency* exists between US_i and US_j **if and only if**

$$\begin{aligned}
&\exists\,(a,b) \in S : \\
&\quad \big[(a,b) \in A(US_i) \times A(US_j) \ \wedge\ \neg\,\text{OBD}(a,b) \ \wedge\ \text{permits}(a,b)\big] \\
&\quad \oplus \\
&\quad \big[(a,b) \in \big(A(US_i) \times R(US_j) \cup R(US_i) \times A(US_j)\big) \ \wedge\ \big(a \approx b \ \vee\ \text{permits}(a,b)\big)\big] \\
&\quad \oplus \\
&\quad \big[(a,b) \in R(US_i) \times R(US_j) \ \wedge\ \text{permits}(a,b)\big]
\end{aligned}$$

In words: A *sequential dependency* between two user stories exists when an activity or reason in one story establishes the system conditions—such as services, configurations, or environments—that are necessary for an activity or reason in the other story to be

meaningfully carried out. This relationship can occur whether the connection is based on actions, justifications, or both, and applies only if there is not already a more specific operation-based (entity-enabling) dependency between them. In other words, sequential dependency describes cases where one story must be completed first to make it possible or logical to implement or perform the other.

Definition 2.6 (**Binary Compositional Dependency - Pairwise**) — Let $E(US)$ be a set of entities which are introduced or manipulated by user story US.
A **Binary Compositional Dependency** exists between US_i and US_j if and only if there exists a pair of entities (e_i, e_j) such that:

$$\begin{aligned} e_i &\in E(US_i), \\ e_j &\in E(US_j), \\ \text{and} \quad \texttt{has-a}(e_i, e_j). \end{aligned}$$

where $\texttt{has-a}(e_i, e_j)$ denotes denotes Meronymy-Holonymy (whole-part) relation, i.e., one of the entity (e_i or e_j) is a component, attribute, or substructure of another.

Definition 2.7 (**Hierarchical Dependency - Pairwise**) — Let $E(US)$ be a set of entities which are introduced or manipulated by user story US.
A **Hierarchical Dependency** exists between US_i and US_j if and only if there exists a pair of entities (e_i, e_j) such that:

$$\begin{aligned} e_i &\in E(US_i), \\ e_j &\in E(US_j), \\ \text{and} \quad \texttt{is-a}(e_i, e_j). \end{aligned}$$

where $\texttt{is-a}(e_i, e_j)$ denotes Hyperonym- Hyponym (type-subtype) relation , i.e. one of the entity (e_i or e_j) is a subtype or specialization of another.

Definition 2.8 (**Alternative Dependency - Pairwise**) — Let U be a set of user stories, and let $US_i, US_j \in U$.

Let $A(US_i)$ and $A(US_j)$ denote the sets of activities associated with US_i and US_j, respectively. Let $R(US_i)$ and $R(US_j)$ denote the sets of reasons associated with US_i and US_j, respectively.

Each activity or reason $a \in A(US_i) \cup R(US_i)$ (and $b \in A(US_j) \cup R(US_j)$) is defined as a pair:

$$a = (\text{Action},\ \text{Entity}), \qquad b = (\text{Action},\ \text{Entity})$$

Define the set:

$$S := \big(A(US_i) \cup R(US_i)\big) \times \big(A(US_j) \cup R(US_j)\big)$$

Let $C(US_i)$ and $C(US_j)$ denote the sets of `contains`-edges for US_i and US_j, where each `contains`-edge is a pair of entities (c, e), with c as a filler for e.

Define the predicates:

- $\approx$: denotes *lexical identity or semantic equivalence* between entity `name` - attributes.
- alternative(c_i, c_j): is true if $c_i \neq c_j$ (i.e., c_i and c_j are different options, which are on the same level).

There exists an **Alternative Dependency** between US_i and US_j if and only if:

$$\exists\,(a,b) \in S,\ (c_i, a.\text{Entity}) \in C(US_i),\ (c_j, b.\text{Entity}) \in C(US_j):$$
$$a.\text{Action.name} = b.\text{Action.name}\ \wedge\ a.\text{Entity.name} \approx b.\text{Entity.name}\ \wedge\ \text{alternative}(c_i, c_j)$$

In words: Two user stories are alternatives if they do the same Action (primary / secondary) on the same Entity (e.g., both "import data"),but each story contains a different filler for that entity (e.g., one imports data in Excel, the other in JSON).

Definition 2.9 (**Self Reflect Dependency**) — Let U be a set of user stories, and let $US \in U$ be a user story.

Let $A(US)$ be the set of activities associated with US, where each activity $a \in A(US)$ is defined as a pair:

$$a = (a.\text{Action.name},\ a.\text{Entity.name})$$

Specifically, *Action* denotes a *GoalAction* and *Entity* denotes a *GoalEntity*.
Define the following:

- requires(a_i.Action.name, a_j.Action.name): The operation described in the `name`-attribute of Action of a_i strictly requires prior implementation of the operation described in the `name`-attribute of Action of a_j, *within the same user story.*
- a_i.Entity.name $\sim$ a_j.Entity.name: There exists a semantic relationship between the `name`-attributes of Entity of a_i and a_j, where $\sim$ indicates any form of semantic relatedness as relevant to the user story context.

A **Self-Reflect Dependency** exists in US **if and only if**

$$|A(US)| > 1\ \ \wedge\ \ \exists\, a_i, a_j \in A(US),\ a_i \neq a_j:\ \ \big[\, a_i.\text{Entity.name} \sim a_j.\text{Entity.name}$$
$$\wedge\ \ \text{requires}(a_i.\text{Action.name},\ a_j.\text{Action.name})\,\big]$$

In words: A user story exhibits Self-Reflect Dependency if it includes more than one activities (primary actions), and there is at least one pair of activities where one activity is required for the execution or meaning of the other within the same story.

For example,
Predecessor:
1. Fall
#G14# As a consumer, I want to view the data package, so that that I can get a sense of whether I want this dataset or not.
#G14# As a Publisher, I want to publish a data package in the UI, so that that it is available and published.

2. Fall
#G14# As a Publisher, I want to sign up for an account, so that that I can publish my data package to the registry and to have a publisher account to publish my data package under.
#G14# As a Publisher, I want to publish a dataset, so that I can view just the dataset with a few people.
Parent:
#G14# As a Publisher, I want to sign up for an account, so that that I can publish my data package to the registry and to have a publisher account to publish my data package under.
#G14# As a Visitor, I want to sign up via github or google, so that that I don't have to enter lots of information and remember my password for yet another website.
Alternative:
#G14# As an Owner, I want to invite an existing user, so that the user can become a member of my publisher.
#G14# As an owner, I want to invite someone using their email to sign up and become a member of my Publisher, so that that they are authorized to publish data packages under my Publisher.

3 Methodology

3.1 Dataset

Which quality criteria have been already checked in the paper of Hofmann?

3.2 PoC

3.3 Evaluation

Bibliography

[1] Sathurshan Arulmohan, Marie-Jean Meurs, and Sébastien Mosser. Extracting domain models from textual requirements in the era of large language models. In *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2023 Companion, Västerås, Sweden, October 1-6, 2023*, pages 580–587. IEEE, 2023. (zitiert auf S. 6)

[2] Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. Rapid quality assurance with requirements smells. *Journal of Systems and Software*, 123:190–213, 1 2017. (zitiert auf S. 4)

[3] Hui Guan, Guorong Cai, and Hang Xu. Automatic requirement dependency extraction based on integrated active learning strategies. *Mach. Intell. Res.*, 21(5):993–1010, 2024. (zitiert auf S. 6, 7)

[4] Lukas Sebastian Hofmann, Alexander Lauer, Jens Kosiol, Arno Kesper, Philipp Wieber, Amir Rabieyan, and Gabriele Taentzer. Detecting redundancies between user stories with graphs and large language models. (zitiert auf S. 5, 6)

[5] Samantha Jiménez, Arnulfo Alanis, Claudio Beltrán, Reyes Juárez-Ramírez, Alan Ramírez-Noriega, and Claudia Tona. USQA: A user story quality analyzer prototype for supporting software engineering students. *Comput. Appl. Eng. Educ.*, 31(4):1014–1024, 2023. (zitiert auf S. 3)

[6] Sen-Tarng Lai. A user story quality measurement model for reducing agile software development risk. *International Journal of Software Engineering & Applications (IJSEA)*, 8, 2017. (zitiert auf S. 3)

[7] Dean Leffingwell and Pete Behrens. A user story primer. 2009. (zitiert auf S. 3, 5, 6, 8)

[8] Juan Li, D. Ross Jeffery, Kam Hay Fung, Liming Zhu, Qing Wang, He Zhang, and Xiwei Xu. A business process-driven approach for requirements dependency analysis. In Alistair Barros, Avigdor Gal, and Ekkart Kindler, editors, *Business Process Management - 10th International Conference, BPM 2012, Tallinn, Estonia, September 3-6, 2012. Proceedings*, volume 7481 of *Lecture Notes in Computer Science*, pages 200–215. Springer, 2012. (zitiert auf S. 8)

[9] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn, E M Van Der Werf, and Sjaak Brinkkemper. Improving user story practice with the grimm method: A multiple case study in the software industry. (zitiert auf S. 4)

[10] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn, E M Van Der Werf, and Sjaak Brinkkemper. Visualizing user story requirements at multiple granularity levels via semantic relatedness. (zitiert auf S. 5)

[11] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn E. M. van der Werf, and Sjaak Brinkkemper. Improving agile requirements: the quality user story framework and tool. *Requir. Eng.*, 21(3):383–403, 2016. (zitiert auf S. 4, 6, 7, 8)

[12] Garm Lucassen, Marcel Robeer, Fabiano Dalpiaz, Jan Martijn E. M. van der Werf, and Sjaak Brinkkemper. Extracting conceptual models from user stories with visual narrator. *Requir. Eng.*, 22(3):339–358, 2017. (zitiert auf S. 5)

[13] Quim Motger and Xavier Franch. Automated requirements relation extraction. 1 2024. (zitiert auf S. 7)

[14] Oemer Uludag, Pascal Philipp, Abheeshta Putta, Maria Paasivaara, Casper Lassenius, and Florian Matthes. Revealing the state of the art of large-scale agile development research: A systematic mapping study. *Journal of Systems and Software*, 194, 2022. (zitiert auf S. 3)

[15] Xiangqian Xu, Yajie Dou, Liwei Qian, Zhiwei Zhang, Yufeng Ma, and Yuejin Tan. A requirement quality assessment method based on user stories. *Electronics (Switzerland)*, 12, 5 2023. (zitiert auf S. 4)

[16] He Zhang, Juan Li, Liming Zhu, Ross Jeffery, Yan Liu, Qing Wang, and Mingshu Li. Investigating dependencies in software requirements for change propagation analysis. (zitiert auf S. 7)